

Low-level optimizations in high-level HDLs: Is there a benefit?

Oliver Keszocze¹[0000-0003-2033-6153], Tjark Petersen²[0000-0002-0239-511X],
Arved Friedemann³[0000-0001-7252-264X], and Matthias Bo
Stuart²[0000-0003-0809-530X]

¹ Computer Engineering, TU Clausthal, Clausthal-Zellerfeld, Germany
`oliver.keszocze@tu-clausthal.de`

² Embedded Systems Engineering, Technical University of Denmark,
Kongens Lyngby, Denmark
`tjape@dtu.dk`, `mbst@dtu.dk`

³ Independent Researcher
Hamburg, Germany
`arved.friedemann@stoppe.de`

Abstract. This paper explores the applicability of functional programming to the design of Application-specific Integrated Circuits (ASICs). We investigate the impact of designing ASICs using high-level, abstract Hardware Description Language (HDL) features versus employing low-level optimizations on the area of the synthesized circuits. The aim is to determine whether using low-level optimizations is beneficial and, if so, whether it is worth the added implementation effort.

To carry out the investigation, we implement an unsigned bit-serial multiply-accumulate (MAC) unit in 16 different configurations using the functional HDL Clash. We make use of both low-level bit-manipulation techniques as well as Clash’s high-level constructs for circuit design.

The experimental evaluation shows that some high-level constructs of Clash have negligible influence on the resulting circuit size, suggesting that using the full power of functional programming is a viable approach to hardware design. To evaluate the impact of the used HDL itself, we also implemented versions of the MAC in Verilog. The experiments clearly show that there seems to be an inherent overhead in using Clash compared to Verilog code written by a seasoned engineer.

1 Introduction and Related Work

A major concern when using higher levels of abstraction for the description of circuits is that the abstractions add logic overhead to the circuit. Very little knowledge exists in the form of measurements or experiments that support or contradict this claim. This paper provides such knowledge by investigating the Application-specific Integrated Circuits (ASICs) implementation of a tiny but non-trivial coprocessor described in a high-level functional programming language with and without various low-level optimizations.

Many different modern Hardware Description Languages (HDLs) have been proposed, many of which are either implemented as a framework or as a Domain-Specific Language (DSL) in an all-purpose programming language (e.g., Chisel and SpinalHDL for Scala [1, 2] or MyHDL and Amaranth for Python [3, 4]) or as an extension of an existing programming language (e.g., Clash [5] for Haskell).⁴ All of them allow describing and/or generating hardware in more flexible and abstract ways than when using Verilog or VHDL.

Chisel has been proven to be a language that is capable of realizing large System-on-Chips (SoCs) via the Rocket Chip project [6]. With the diplomacy framework, there is a tool that allows to negotiate parameters between individual Chisel generators [7] while staying entirely in the host language Scala, allowing for a highly integrated design process. There is very little literature on Chisel being used for small designs that perform low-level bit operations (see [8] for a paper presenting initial ideas on smaller designs and building blocks and [9] for a study similar to the one presented in this paper but targeting FPGAs).

The programming style for Chisel is, as in the host language Scala, a mixture between imperative and functional programming. The language Clash is fully embracing the functional approach: based on the pure functional programming language Haskell, it greatly raises the level of abstraction that can be used when describing hardware in it. According to the documentation [10], Clash will not introduce any logic overhead:

What is always important to remember is that [...] the compiler will *never* infer/invent more logic than what is specified in the circuit description. (emphasis theirs)

The high level of abstraction in Clash code, and to a certain extend in Chisel as well, may give rise to the impression that actually a high-level synthesis (HLS) is carried out. This is not the case. Consequently, this paper does *not* deal with the issue of overhead introduced by HLS with respect to RTL.

Although Clash is not as widely used as Chisel, it has seen some interest in implementing processors in it. There are fully verified RISC-V implementations [11] and even a textbook on retrocomputing that re-implements multiple classic processors [12].

The authors are not aware of any publications describing the usage of Clash to implement small designs using manual low-level optimizations.

The design used as a working example in this paper is an unsigned integer bit-serial multiply-accumulate (MAC) unit written in Clash. The aim is to implement it as area-efficient as possible. This paper will investigate how different levels of abstraction will impact the size of the synthesized design and will also compare these findings against hand-written Verilog versions of the MAC to investigate the impact of the HDL used.

⁴ The interested reader is referred to the **awesome-hdl** GitHub repository at <https://github.com/drom/awesome-hdl> that hosts a curated list of “amazingly awesome hardware description language projects.”

2 The Clash HDL

Clash is a functional programming language for circuit design based on the functional programming language Haskell. It models combinatorial circuits as pure functions of type $f : A \rightarrow B$ where A and B are finite sets. The corresponding Clash notation is `f :: A -> B`. Very few restrictions apply in order for Clash to be able to synthesize code into either Verilog, SystemVerilog or VHDL; mainly that every data type needs to be finite and that recursions are statically known to terminate.

A simple implementation of computing the parity bit of a vector of `n` bits in Clash is given by

```
parityBit :: Vec n Bit -> Bit
parityBit v = fold xor v
```

The `fold` function creates a tree structure of `xor` operations of logarithmic depths in order to minimize the delay. One can see that Clash uses a high level of abstraction that allows the developer to clearly express the intent and, hence, focus on the design and not technical details.

Sequential circuits are modeled using the data type `Signal dom a`. It represents an infinite stream of values of type `a` in some clock domain `dom`.⁵ When using this data type, Clash will generate the required registers to store the values. Clash will also generate clock, reset and enable signals based on the domain `dom`. When working with multiple clock domains, Clash effectively prevents accidental clock domain crossings.

Clash comes bundled with data types that cover the common use cases: bit-vectors of arbitrary size n (`BitVector n`), (Un-)Signed integers of bit-width n (`Unsigned n/Signed n`) as well as vectors of length n storing arbitrary data of type `a` (`Vec n a`). Values that come with a valid status can be modeled via the `Maybe a` data type where `Just 4` is a valid integer and `Nothing` indicates the absence of a valid value. This data type prevents the accidental use of bits that do not contain valid data.

Clash is very strict with respect to type checking. Even if two types have the same bit-representation, they can not be assigned to each other. The designer has to explicitly cast between the types.

As the types are checked at compile time, no logic overhead is generated in the Verilog or VHDL files.

There are two main approaches when implementing hardware in Clash as discussed in the following two sections.

2.1 Mealy Machines

One method for implementing sequential circuits is to model them as a *Mealy Machine* [13]. This classical approach makes use two functions T and G , the

⁵ Concrete types in Clash have to start with a capital letter. A small letter indicates a placeholder for types, similar to generics in Scala.

transition and output function, respectively. In Clash, a coalesced function that combines both functions in one is employed instead. Given a function f of type $f :: s \rightarrow i \rightarrow (s, o)$ mapping the current state of type s and the current input of type i to a tuple consisting of the new state and the Mealy Machine’s output of type o . Clash provides the `mealy` function of type

```
mealy :: (s -> i -> (s, o)) -> s -> Signal dom i -> Signal dom o
```

that transforms such a coalesced transition/output function f together with the initial state to a circuit implementing the Mealy machine. As an example consider the following Mealy machine that detect runs of consecutive 1’s of length greater than three:⁶

```
detectRun :: Int -> Bit -> (Int, Bool)
detectRun count b =
  if b == 1
  then (count + 1, count + 1 > 3)
  else (0, False)

circuit :: Signal dom Int -> Signal dom Bool
circuit = mealy detectRun 0
```

No manual definition of registers that store the internal state is necessary. Note that f is a purely combinatorial function/circuit.

2.2 State Monad-based Hardware

Being a functional programming language, Haskell and, by extension, Clash, does not have a native concept of *mutable state*. This is why the state was explicitly passed to the Mealy machine in the previous section. To avoid this and to write code that looks like the “conventional” imperative programming style, Haskell/Clash makes use of the so-called *state monad* and provides a corresponding `mealyS` function.

We omit the technical details here and only point out that using a state monad-based programming approach allows describing hardware in a way that often more closely resembles the algorithmic description of the algorithm the hardware should implement, making the code easier to understand and maintain.

A state monad-based implementation of the previous example is given by the following code.

```
detectRunMonad :: Int -> State Int Bool
detectRunMonad i = do
  if i == 1
  then (modify' (+1))
  else (put 0)
```

⁶ All shown codes examples are simplified or/and have parts omitted for didactic purposes. The full code of the presented MAC is available on GitHub.

```

cnt <- get
return $ cnt > 3

circuitMonad :: Signal dom Int -> Signal dom Bool
circuitMonad = mealyS detectRunMonad 0

```

3 Sequential Arithmetic

Digital multipliers are well-studied circuits with a great amount of variation in designs and implementations seeking to optimize for one or more of area, speed, or power-consumption [14]. Given an unsigned n -bit multiplicand X , and an unsigned m -bit multiplier Y , their product is given by as

$$X \cdot Y = \left(\sum_{i=0}^{n-1} x_i 2^i \right) \cdot \left(\sum_{j=0}^{m-1} y_j 2^j \right) = \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} x_i \cdot y_j \cdot 2^{i+j}. \quad (1)$$

The bit-wise product $x_i \cdot y_j$, is realized by a logic and-operation, while the scaling 2^{i+j} is a matter of bit-shifting or multiplexing. In a low-resource implementation, registers are used to hold each of the factors and the partial product. A single full-adder is used to iteratively calculate the partial sum of a single bit of the partial product, a bit-wise product of one bit from each operand, and the carry from the previous iteration.

This is illustrated in Algorithm 1, where the outer loop ranges over the bits of Y (we refer to this as a “round”) and the inner loop ranges over the bits of X . The bit-wise product $x_i \cdot y_j$ is computed by an and-operation. The new value of the current product bit is then the sum of the bit-wise product, the current product bit and the incoming carry.

After each round, the carry is propagated to the next product bit; which is possible because that bit is known to be zero. This is done by the **endRound** method (line 10). The calls to **advanceX/Y** (lines 8 and 11) will be explained in Section 4.4. For now, consider them to be no-operations.

An exemplary calculation for $X = 6$ and $Y = 3$ is presented in the left part of Table 1.

The “moving parts”, i.e., steps that render themselves amendable for design choices, will be discussed in the next section. When applicable, those moving parts are highlighted in blue in Algorithm 1.

The sequential accumulating step following the sequential multiplication does not introduce further optimization potential and, hence, is omitted for readability.

4 Implementation

The MAC’s implementation features a n -bit and an m -bit input for the operands, a $n + m$ -bit input for manually (pre-)setting the accumulator’s value and enable

Data: Multiplicand and multiplier x, y of width n and m , respectively

```

1 carry, product  $\leftarrow 0$ 
   // m partial products/rounds
2 for roundCtr  $\leftarrow 0$  to  $m - 1$  do
   // n bit positions within X
3   for xCtr  $\leftarrow 0$  to  $n - 1$  do
     // The arithmetic part
4      $a \leftarrow \text{getXBit}(xCtr) \wedge \text{getYBit}(\text{roundCtr})$ 
5      $p \leftarrow \text{getProdBit}(\text{roundCtr}, xCtr)$ 
6      $(\text{carry}, s) \leftarrow \text{FullAdder}(a, p, \text{carry})$ 
     // Updating the product
7      $\text{setProdBit}(\text{roundCtr}, xCtr, s)$ 
8      $\text{advanceX}()$ ;
9   end
10   $\text{endRound}(\text{carry}, xCtr, \text{roundCtr})$ 
11   $\text{advanceY}()$ 
12   $\text{carry} \leftarrow 0$  // Reset carry
13 end
14 return product

```

Algorithm 1: Abstract view on the sequential multiplication.

	index-based						rotation-based					
	x	y	c_{in}	$product$	c_{out}	$product'$	x	y	c_{in}	$product$	c_{out}	$product'$
P	<u>11</u> 0	<u>11</u>	0	0000 <u>0</u>	0	00000	<u>11</u> 0	<u>11</u>	0	0000 <u>0</u>	0	00000
P	<u>11</u> 0	<u>11</u>	0	0000 <u>0</u>	0	00010	01 <u>1</u>	<u>11</u>	0	0000 <u>0</u>	0	00001
P	<u>11</u> 0	<u>11</u>	0	00 <u>0</u> 10	0	00110	10 <u>1</u>	<u>11</u>	0	1000 <u>0</u>	0	10001
E			0	0 <u>0</u> 110	0	00110			0	1100 <u>0</u>	0	11000
P	<u>11</u> 0	<u>11</u>	0	001 <u>1</u> 0	0	00110	<u>11</u> 0	<u>01</u>	0	0001 <u>1</u>	0	00011
P	<u>11</u> 0	<u>11</u>	0	00 <u>1</u> 10	1	00010	01 <u>1</u>	<u>01</u>	0	1000 <u>1</u>	1	10000
P	<u>11</u> 0	<u>11</u>	1	0 <u>0</u> 010	1	00010	10 <u>1</u>	<u>01</u>	1	0100 <u>0</u>	1	01000
E			1	<u>0</u> 0010	1	10010			1	0010 <u>0</u>	1	00101
E'				00010					1	1001 <u>0</u>		

Table 1: The computation carried out by Algorithm 1 for $X = 6 = 110_2$ and $Y = 3 = 11_2$ for the conventional, indexing-based data access (left) and the rotation-based data access (right). The bits used in each step are underlined. To help to understand the rotation-based method, the least-significant bits of x and the product are highlighted in bold. The type of operation carried out in each row corresponds to the algorithm as follows: **P**roduct step (lines 3–8) and **E**nd of round step (lines 10–12).

```

1  macMealy st@State{stage=Ready} Input{Nothing, Nothing}
2    = (st, getOutputs st)
3  macMealy st@State{stage=Ready} Input{Nothing, Just newAcc}
4    = (st', getOutputs st')
5    where st ' = st{accumulator=newAcc}
6  macMealy st@State{stage=Multiplying, ..} Input{Nothing, Nothing}
7    = (st', getOutputs st')
8    where st ' = --
9  -- more cases following (omitted here)

```

Listing 1: Clash code using a Mealy machine to implement the MAC. Shown is an excerpt of the multiplication part.

bits for initiating a MAC operation or setting the accumulator, respectively. The design outputs two $n + m$ -bit outputs for the last product and the accumulator with corresponding valid bits.

The rest of this section describes the four design choices (“moving parts”) investigated in this paper.

4.1 Mealy Machine vs. State Monad

This design choice is the most abstract one: should the designer opt for the well-known approach of implementing a Mealy machine or should a more abstract approach be followed.

Listing 1 shows a simplified code excerpt of how the corresponding Mealy machine could be implemented in Clash. Each state-input pair must be matched and then used to compute the next state and the output.

Contrast this with the code shown in Listing 2. The state monad-based approach allows organizing the code in a fashion that it resembles an imperative programming language. The conditional setting of the accumulator value in Line 3 and the general linear flow give the impression of an imperative programming language. The MAC’s state is handled implicitly; nothing has to be passed on explicitly. This can lead to code that is easier to read and might come more natural to implement designs in.

We posit that using the state monad-based approach is more straight-forward for designers used to imperative languages and, hence, might lead to shorter design cycles.

4.2 Data Type of the Product

There are two obvious ways to store the product bits: Either as a `BitVector n`, that is, n consecutive bits or using the type `Vec n Bit`; a vector that has n entries of type `Bit`. Although these types look very similar, `Vec` is more general as it allows to store arbitrary data types, not just `Bits`, and comes with a richer

```

1  macMonad accumulateFun multiplyFun Input{values, newAcc} = do
2    -- conditionally set the accumulator to a new value
3    case newAcc of
4      (Just acc) -> modify' (\s -> s{accumulator=bitCoerce acc})
5      Nothing -> pure ()
6
7    -- start a new multiplication or continue with current computation
8    case values of
9      Just (x,y) -> modify' (startMulState x y)
10     Nothing -> do
11       stage <- gets stage
12       case stage of
13         Ready -> pure () -- do nothing
14         Multiplying -> modify' multiplyFun
15         Accumulating -> modify' accumulateFun
16
17    -- retrieve the current output
18    gets extractOutput

```

Listing 2: Excerpt of the clash code implementing the MAC using the state monad-based approach.

Application Programming Interface (API) that, e.g., provides many folding operations that generate logarithmic reduction trees. Both types should be reduced to the same Verilog type when using `Vec n Bit`. Hence, we expect that there should be no significant difference in the resulting circuits allowing designers to choose the data type/API that suits them best or is most descriptive.

4.3 Data Type of the Counters

Clash provides the type `Index n` consisting of the integers $[0, n - 1]$ represented in the usual binary encoding using $\lceil \log_2(n) \rceil$ bits. This is the natural high-level choice for the counters `roundCtr` and `xCtr` (see Algorithm 1, lines 2 and 3).

As incrementing an integer requires a (possibly specialized) adder, we implement another type of counter based on one-hot encoding. For this, a `BitVector n` is used that starts with the least significant bit set to one and all other bits set to zero. Each time the counter is increased, the set bit is shifted left by one until it reaches the most significant bit. This implementation does not need any adder hardware but requires designers to implement the overflow logic on their own and, therefore, is a rather low-level approach; even though this can effectively be hidden from the end-user by providing utility functions. For larger bit widths, we expect this implementation to become too unwieldy as it requires n bits instead of the $\lceil \log_2(n) \rceil$ bits used by `Index`.

4.4 Accessing and Updating Data

Accessing and updating the product and reading the current position within X and Y can be carried out in different ways. Algorithm 1 uses functions allow for a different implementation (see lines 5 and 7).

One approach is to take the counter values and compute an index to access the product, e.g., $product_{roundCtr+xCtr}$ accesses the currently used product bit.⁷ This is a high-level approach to implementing the MAC in an intuitive fashion that closely resembles programming with arrays in regular programming languages. We call this the *index-based* approach.

Reading and writing to arbitrary positions within the product requires (de-)multiplexing. To avoid this, another approach is investigated in this paper: replacing arbitrary indexing by indexing to the least-significant bit of a shifted product, e.g., $x[3] \triangleq (x \gg 3)[0]$. To avoid having to support shifting all values between 1 and $n - 1$, we only shift by 1 and store the updated value. This approach requires special care of how the product register is rotated as it needs to be reset after each round. As the patterns for X repeat, we rotate this value instead of shifting in zeros. At the end of a round, the product needs to be shifted back by $n - 1$ to start the next round at the next-higher bit.

Implementing Algorithm 1 this way yields very difficult to understand Clash code. Designers should only opt to implement a design this way if there is a clear benefit with respect to non-functional properties, such as the circuit’s area and make sure to “hide” it using a reasonable function names. We call this approach *rotation-based*.

The right part of Table 1 illustrates this approach for the multiplication of $X = 6$ and $Y = 3$.

These two approaches lead to the implementations of the functions highlighted in blue in Algorithm 1 as shown in Table 2. The **endRound** step becomes quite complicated for the rotation-based approach as shown in Algorithm 2.

operation	index-based	rotation-based
getX/YBit	$x/y[ctr]$	$x/y[0]$
getProdBit	$product[roundCtr+xCtr]$	$product[0]$
setProdBit	$product[roundCtr+xCtr]=s$	$product[0]=s$
advanceX	–	$x = \text{rotate } x \ 1$
advanceY	–	$y = y \gg 1$
endRound	$product[roundCtr+xCtr+1]=c$	see Algorithm 2

Table 2: Implementations of the “moving parts” depending on the choice of the accessing/modification method.

⁷ For the one-hot encoding approach, the position of the set bit has to be translated into a number first.

```

1 product ← rotate product 1
2 product[0] ← c
3 if after last round then
4   | product ← rotate product 1
5 else
6   | product ← rotateBackwards product (n − 1)
7 end

```

Algorithm 2: The complex `endRound` step when using the rotation-based approach.

4.5 Overview of the Designs

The choices described above give rise to 16 different designs per choice of bit-width in total as the following decisions can be made:

- Mealy-machine-based or state monad-based modeling
- Data type of the product: `BitVector` or `Vec`
- Data type of the counters: one-hot encoding or using an off-the-shelf `Index`
- Method for updating the product: index-based or rotation-based

The code necessary to generate all designs is publicly available on GitHub⁸.

4.6 Testing the Clash Designs

Modern HDLs usually also come with sophisticated testing and/or verification frameworks. Chisel, for example started with the rather simple `chiseltest` framework which was later extended by ChiselVerify [15] to support, amongst others, Constrained Random Verification (CRV) [16].

While Clash supports the automatic generation of classical Verilog test benches, we agree with Érdi, quoting from [12, Page 24]:

“One common trait among FPGA vendor tools is that they range from horribly painful to painfully horrible. In this book, interaction with the FPGA toolchain is restricted to a minimum by writing everything we can in Clash”

Clash designs are valid Haskell code and, therefore, can be tested using any of the many software testing frameworks available for Haskell.⁹ The program carrying out the tests is then similar to running a Verilator¹⁰ simulation.

A particularly interesting type of testing is *property-based testing* [17, 18]. This approach differs from classical unit testing in that it does not rely on pre-specified inputs and the corresponding golden answers but allows to specify

⁸ <https://github.com/keszocze/mac-clash-iwsbp26>

⁹ The authors are fully aware that this only covers functional testing and that it relies on the assumption of the Clash compiler being bug-free.

¹⁰ <https://www.veripool.org/verilator/>

properties (such as $\forall x, y : \text{MAC}(x, y, acc) = x \cdot y + acc$) that are automatically validated by generating random values and checking whether the property holds.

To prune the design space, we only investigate design with $n = m$. For bit-widths $n > 8$, the designs have been tested with 200 randomly generated inputs. For bit-widths n with $n \leq 8$, all possible 2^{n+n} inputs have been simulated, effectively fully verifying the smaller designs. We do expect the test results to also hold for cases where $n \neq m$.

5 Experimental Evaluation

5.1 Verilog Designs

We also implemented the MAC unit in Verilog in order to investigate whether differences in the area of synthesized designs stem from different implementation choices or the used HDL. We are further interested to see whether there is a general overhead introduced by either of the languages.

The architectural variants – indexing vs. rotating and base-2 counters vs. one-hot counters – were also directly implemented in Verilog code. The interface’s timing behavior was matched between the Clash and Verilog implementations, and was verified using simulations of the hand-written Verilog code together with the Verilog code compiled from the Clash implementation. To an external observer, all implementations of the circuit would be identical.

5.2 Experimental Setup

Both Clash-derived and hand-written Verilog code was passed through the initial steps of the default LibreLane flow for chip design using the SkyWater 130 nm PDK, including linting, error-checking, and synthesis using the open-source tool Yosys. The synthesis step produces a netlist of standard cells defined in the process design kit (PDK). Standard cells correspond to memory elements in the form of D-flipflops and logic elements in the form of logic gates such as “nand” and “nor” as well as slightly more advanced logic operations on a few bits. The PDK describes the function of the standard cells as well as their individual area on a chip. For comparing the various implementations, the total area of the synthesized circuit as well as the area of the sequential (memory) and logic parts is considered individually.

The decision to stop after synthesis was made deliberately. Many additional steps are needed to produce a functioning chip, but none of these steps take the Clash-derived or hand-written Verilog files as input. They are rather based off the synthesized netlist or other representations derived from the netlist. Each of these steps may introduce changes to the circuits’ areas due to the physical constraints of semiconductors and the manufacturing steps, but as these changes are not directly related to the initial description of the circuit, they are considered confounding factors better left out of the comparison.

We ran the flow for the 27 bit-widths $n = 2, 3, \dots, 16, 20, 24, \dots, 64$ and all 16 configurations as described in Section 4, effectively performing a full Design Space Exploration (DSE).

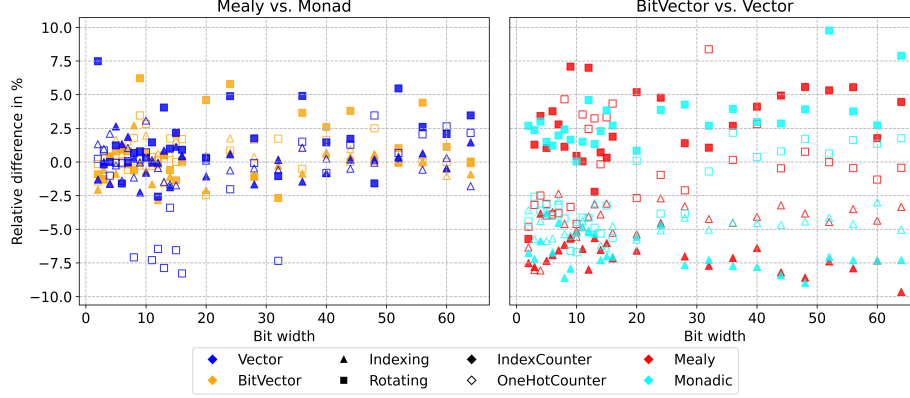


Fig. 1: Clash design choices with low influence on circuit sizes, left: Mealy machine vs. state monad, right `BitVector` vs. `Vec`. Positive values indicate larger area for Mealy (left) and `BitVector` (right) based implementations. The legend shows individual factors by color, shape, and filled/hollow markers.

5.3 Evaluating the Design Choices

In this section, we will discuss the relative difference between circuits varying in only one aspect, i.e. a single design choice (see Section 4). More concretely, given two different versions v_1 and v_2 of the MAC unit, we compute

$$\frac{\text{size}(v_1) - \text{size}(v_2)}{\text{mean}(\text{size}(v_1), \text{size}(v_2))},$$

i.e. the difference of all sizes for each design choice divided by the mean over all sizes. The relative difference is shown as a function of the bit-width.

Variations within the Clash Designs Starting with the language specific variations of the Clash implementations, specifically expressing the circuits as a Mealy machine as opposed to a state monad (see Fig. 1, left) and using the `BitVector` type as opposed to a `Vector` of bits type, minimal impact is seen on the circuits' areas (see Fig. 1, right). This indicates that little to no overhead is associated with either of these variations. Consequently, for the investigated circuits, designers can use any of these approaches with little concern for the size of the synthesized circuit.

Moving into the architectural variations, comparing indexing into vs. rotating the registers, a clear difference is seen (see Fig. 2, left). This is to be expected, as N -input multiplexers, required for indexing, typically are implemented as a tree of 2- or 3-input multiplexers giving an area scaling of $\mathcal{O}(N \log_2(N))$. This should be compared to the variant with rotating (barrel-shift) registers that only requires $\mathcal{O}(1)$ -sized logic to control the shifting. The results show this trend. Two groupings are seen corresponding to the other architectural variation,

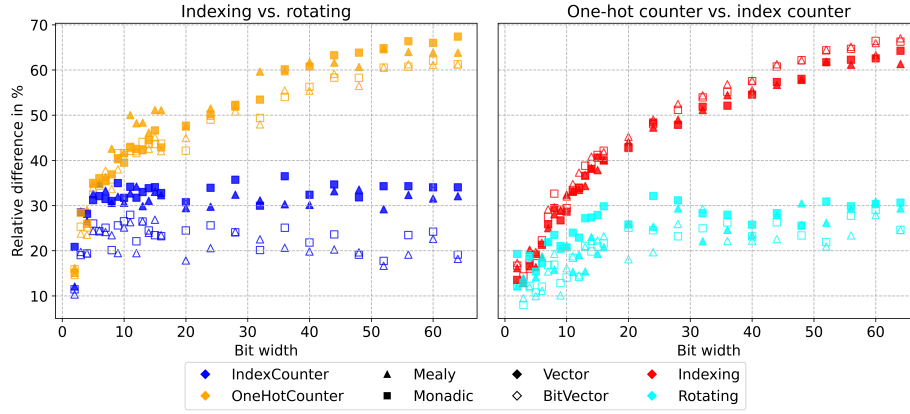


Fig. 2: Clash design choices with strong influence on circuit sizes, left: Indexing vs. rotating accesses, right: One-hot vs. index counters. Positive values indicate larger area for indexing (left) and one-hot counter (right) based implementations.

where one grouping corresponds to the variation using base-2 counters, and the other grouping corresponds to using one-hot counters. Considering the sequential and combinatorial (logic) areas, zero difference is seen in the sequential areas indicating that exactly the same number of bits is stored in registers for both multiplexing and rotating, which is to be expected, whereas the difference seen in total area is matched by the difference in logic area, which includes multiplexers.

Considering the comparison between counter types, again a clear difference is seen (see Fig. 2, right). Using base-2 counters (i.e., [Index](#)) requires registers of $\lceil \log_2(N) \rceil$ bits and having adders of the same width, whereas using one-hot counters requires a shift-register and $\mathcal{O}(1)$ logic to control this register. Looking at the area of sequential standard cells (flipflops), this difference in register width is clearly seen and it is identical for all variations (all series have near-identical relative differences). Looking at the difference in logic area, again two groupings are seen corresponding to indexing vs. rotating. The circuits with base-2 counters have lower area than those with one-hot counters, and for the first grouping corresponding to rotating the difference is roughly constant, whereas it depends on the bit-width for the second grouping corresponding to multiplexing. This is as expected.

Variations within the Verilog Designs Comparing architectural variations in the Verilog implementations shows similar trends as for the implementations written in Clash: Indexing takes up larger area than rotating, albeit less pronounced than in the Clash case (see Fig. 3, left), and one-hot counters take up larger area than base-2 counters (see Fig. 3, right) with all the difference being in the logic for multiplexing vs. rotating, and the counter variations having both different sequential and logic areas, as expected.

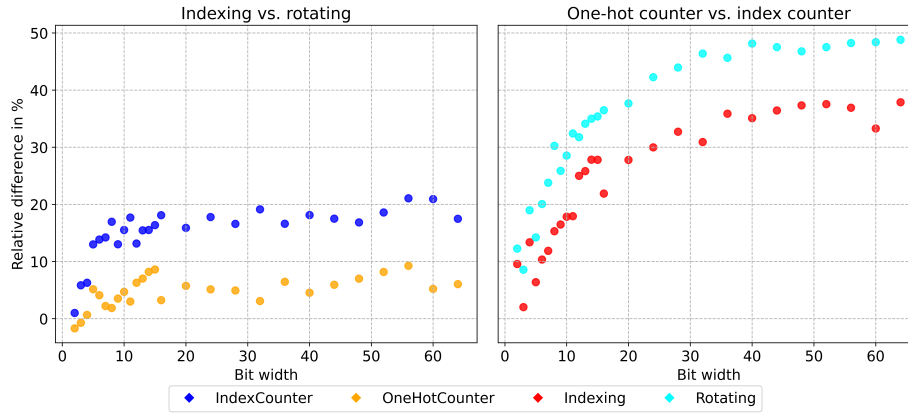


Fig. 3: Design choices in Verilog with a large influence on the resulting circuit size, left: indexing vs. rotating accesses, right: One-hot vs. index counters. Positive values indicate larger area for indexing (left) and one-hot counter (right) based implementations.

Clash vs. Verilog Since little variation was seen between the Mealy machine and state monad-based approach and between using either `BitVector` or `Vector` for the Clash implementations, the hand-written Verilog is compared to the Clash-implementations using a Mealy machine and the `Vector` type.

The area of the hand-written Verilog circuits is lower than that of the Clash implementations in practically all cases (see Fig. 4). The difference, however, varies across the four architectural variations. The largest difference is seen for the circuits using multiplexing. As previously described, all circuits have identical behavior on the interface, therefore it can be concluded that the hand-written Verilog implementations produce netlists of fewer or smaller standard cells than the Clash implementations. This corresponds to a reduced logic complexity, and while the details in why these differences arise are yet to be investigated, it is a strong indication that either the Clash implementation or the Clash compiler results in higher complexity for the same operation than a direct implementation in a lower-level language such as Verilog. Interestingly, both for the area of sequential and logic cells, a dependence is seen on the bit-width with Verilog requiring increasingly less area of both types with increasing bit-width. This indicates that the Clash implementations both have a larger state and a higher logic complexity.

6 Conclusion & Outlook

While the results shown in the previous section indicate smaller state and lower logic complexity of hand-written Verilog compared to the Clash implementations, other factors not included in the present study need to be considered

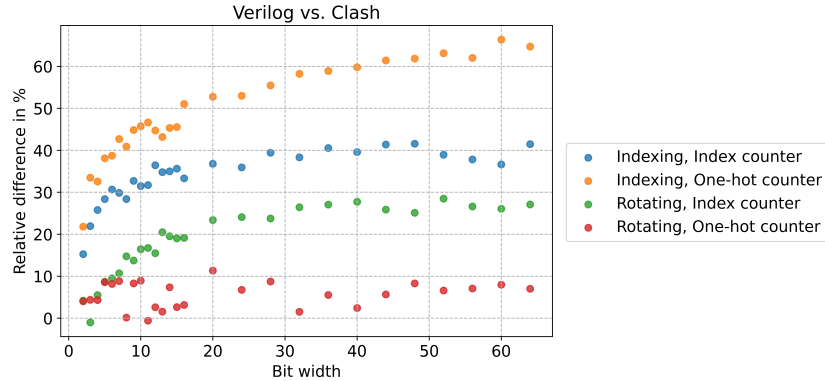


Fig. 4: Size comparison of hand-written Verilog to Clash. Positive values indicate larger area for Clash-implementations.

before making a conclusion about the benefits of one language over another. For one, while area is an important aspect of chip design, other aspects such as latency, throughput, power, and energy consumption are also highly important metrics. For a study as the one presented here with more than a thousand synthesized circuits, the area of the synthesized netlist is the simplest to extract, and given the stated differences in state size and logic complexity, it would be highly unexpected to see opposing trends for area, latency, throughput, or power consumption.

However, these “hard” metrics are not always the most important ones in the real world. For example, the chip design tools tend to create a 2-D bounding box containing the interface on the periphery and the functionality (standard cells) inside the box. For simple circuits with a “large” interface - measured as the total number of bits - the logic complexity may be of secondary importance as the size of the bounding box will be decided by the interface size, while the circuit may only take up a small fraction of the box’s area. In such cases, the agility in development cycles offered by higher-level languages may be preferable over having closer-to-optimal designs from the start. It does not seem farfetched to envision a design flow starting from descriptions using high-level languages in an agile workflow followed by profiling and targeted low-level optimizations or even re-implementations in low-level languages where needed. Further study is needed to evaluate the efficacy of such design flows.

As additional future work, we intend to extend our comparison to the Chisel HDL (similar to [9]) and Amaranth. We will also try to match the areas obtained from the hand-written Verilog code to see how much effort is necessary to become truly competitive with respect to area. We also intend to compare the result obtained by the open-source tools to commercial tools.

References

- [1] Jonathan Bachrach et al. “Chisel: Constructing Hardware in a Scala Embedded Language”. In: *Design Automation Conference*. June 2012.
- [2] *SpinalHDL*. Version v1.10.2a. July 4, 2024. URL: <https://github.com/SpinalHDL/SpinalHDL>.
- [3] J. Decaluwe. “MyHDL: A Python-Based Hardware Description Language.” In: *Linux Journal* 127 (2004). ISSN: 1075-3583.
- [4] *Amaranth HDL*. Version 0.5.8. Amaranth HDL, Aug. 15, 2025. URL: <https://github.com/amaranth-lang/amaranth>.
- [5] Christiaan Baaij et al. “ClaSH: Structural Descriptions of Synchronous Hardware Using Haskell”. In: *Euromicro Conference on Digital System Design*. Sept. 2010. ISBN: 978-1-4244-7839-2.
- [6] Krste Asanović et al. *The Rocket Chip Generator*. UCB/EECS-2016-17. University of California at Berkeley, Apr. 15, 2016.
- [7] Henry Cook, Wesley Terpstra, and Yunsup Lee. “Diplomatic Design Patterns: A TileLink Case Study”. In: *Workshop on Computer Architecture Research with RISC-V*. Oct. 17, 2017.
- [8] Martin Schoeberl et al. “Scala Defined Hardware Generators for Chisel”. In: *Microprocessors and Microsystems* 117 (Sept. 1, 2025). ISSN: 0141-9331.
- [9] Paul Lennon and Richard Gahan. “A Comparative Study of Chisel for FPGA Design”. In: *Irish Signals and Systems Conference*. June 2018.
- [10] QBayLogic B.V. *Clash Tutorial*.
- [11] *Where Lions Roam: RISC-V on the VELDT*. Standard Semiconductor, Aug. 3, 2024. URL: <https://github.com/standardsemiconductor/lion>.
- [12] Gergő Érdi. *Retrocomputing with Clash: Haskell for FPGA Hardware Design*. Sept. 12, 2021.
- [13] George H. Mealy. “A Method for Synthesizing Sequential Circuits”. In: *The Bell System Technical Journal* 34.5 (Sept. 1955). ISSN: 0005-8580.
- [14] Miloš D. Ercegovic and Tomás Lang. *Digital Arithmetic*. June 10, 2003. ISBN: 978-1-55860-798-9.
- [15] Andrew Dobis et al. “ChiselVerify: An Open-Source Hardware Verification Library for Chisel and Scala”. In: *IEEE Nordic Circuits and Systems Conference*. Oct. 2021.
- [16] Ashok B. Mehta. “Constrained Random Verification (CRV)”. In: *ASIC/SoC Functional Design Verification: A Comprehensive Guide to Technologies and Methodologies*. Ed. by Ashok B. Mehta. 2018. ISBN: 978-3-319-59418-7.
- [17] George Fink and Matt Bishop. “Property-Based Testing: A New Approach to Testing for Assurance”. In: *SIGSOFT Softw. Eng. Notes* 22.4 (July 1, 1997). ISSN: 0163-5948.
- [18] Harrison Goldstein et al. “Property-Based Testing in Practice”. In: *International Conference on Software Engineering*. Apr. 12, 2024. ISBN: 979-8-4007-0217-4.